Edge Intelligence Lab

Video Classification Task

25MML0016

Rushikesh Rahul Nirmal

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input,
decode_predictions

# Load pretrained model
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        raise ValueError("Cannot open video")

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    step = max(1, total_frames // max_frames) if total_frames > 0 else 1

    for i in range(total_frames):
        ret, frame = cap.read()
        if not ret:
            break

        if i % step == 0:
            frame = cv2.resize(frame, (224, 224))
```

```
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

        if len(frames) >= max_frames:
            break

    cap.release()
    return np.array(frames)


def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames = preprocess_input(frames)
    predictions = model.predict(frames, verbose=0)

    avg_pred = np.mean(predictions, axis=0)
    top_preds = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    print("\nTop Predictions:")
    for label in top_preds[0]:
        print(f"{label[1]} : {label[2]*100:.2f}%")

    print("\nFinal Prediction:", top_preds[0][0][1])


video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"
classify_video(video_path)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0
_224.h5
14536120/14536120 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Downloading data from
https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_in
dex.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Top Predictions:
cab : 17.92%
traffic_light : 6.14%
streetcar : 5.89%
jinrikisha : 3.63%
limousine : 3.56%
```

```
Final Prediction: cab
```

```python
# ==============================

from google.colab import drive
drive.mount('/content/drive')

# ==============================

!pip install -q tensorflow opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input,
decode_predictions

# ==============================

print("Loading MobileNetV2 model...")
model = MobileNetV2(weights='imagenet')
print("Model loaded successfully!\n")

# ==============================

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames)

    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)
```

```
        count += 1
        if len(frames) >= max_frames:
            break

    cap.release()
    return np.array(frames)

video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"


extracted_frames = extract_frames(video_path)

if len(extracted_frames) == 0:
    raise ValueError("No frames extracted. Check video path.")

print("Extracted Frames Shape:", extracted_frames.shape)
print("Data Type:", extracted_frames.dtype)

# ==============================

plt.figure(figsize=(15,5))
for i in range(min(5, len(extracted_frames))):
    plt.subplot(1,5,i+1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i+1}")
    plt.axis("off")
plt.show()

# ==============================
print("\nTop-left 5x5 Pixel Region of First Frame:")
print(extracted_frames[0, 0:5, 0:5, :])

# ==============================

normalized_frames = extracted_frames / 255.0

print("\nNormalized Range:")
print("Min:", normalized_frames.min())
print("Max:", normalized_frames.max())

# ==============================

processed_frames = preprocess_input(extracted_frames)

# ==============================

print("\nRunning predictions...")
```

```python
predictions = model.predict(processed_frames, verbose=0)

# Average predictions across frames
avg_prediction = np.mean(predictions, axis=0)

# Decode top 5 predictions
top_predictions = decode_predictions(np.expand_dims(avg_prediction, axis=0),
top=5)

print("\nTop 5 Predictions:")
for pred in top_predictions[0]:
    print(f"{pred[1]} : {pred[2]*100:.2f}%")

print("\nFinal Prediction:", top_predictions[0][0][1])
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
Loading MobileNetV2 model...
Model loaded successfully!

Extracted Frames Shape: (20, 224, 224, 3)
Data Type: uint8
```

```
Top-left 5x5 Pixel Region of First Frame:
[[[ 60  67  53]
  [ 57  63  50]
  [125 131 119]
  [103 108 101]
  [ 99 103  98]]

 [[ 55  61  48]
  [ 60  66  53]
  [ 83  89  77]
  [ 88  93  87]
  [121 126 120]]

 [[ 48  54  41]
  [ 62  67  55]
  [ 68  74  63]
  [ 80  85  79]
  [103 108 102]]
```

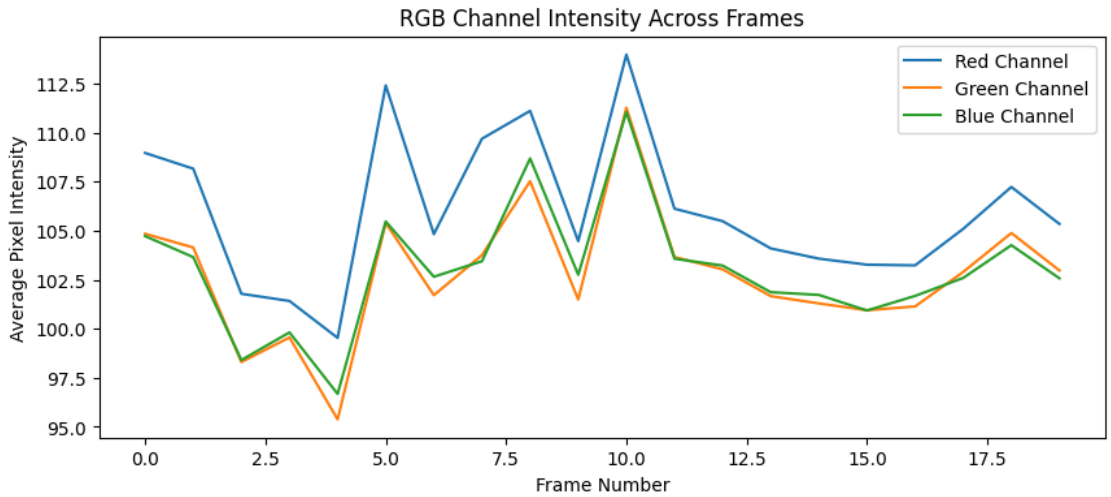
```
 [[ 45  51  38]
  [ 50  56  43]
  [ 60  66  54]
  [ 81  86  80]
  [ 76  81  75]]

 [[ 43  49  36]
  [ 44  47  35]
  [ 50  52  41]
  [102 107 101]
  [ 74  78  73]]]

Normalized Range:
Min: 0.0
Max: 1.0

Running predictions...

Top 5 Predictions:
cab : 17.92%
traffic_light : 6.14%
streetcar : 5.89%
jinrikisha : 3.63%
limousine : 3.56%

Final Prediction: cab
```

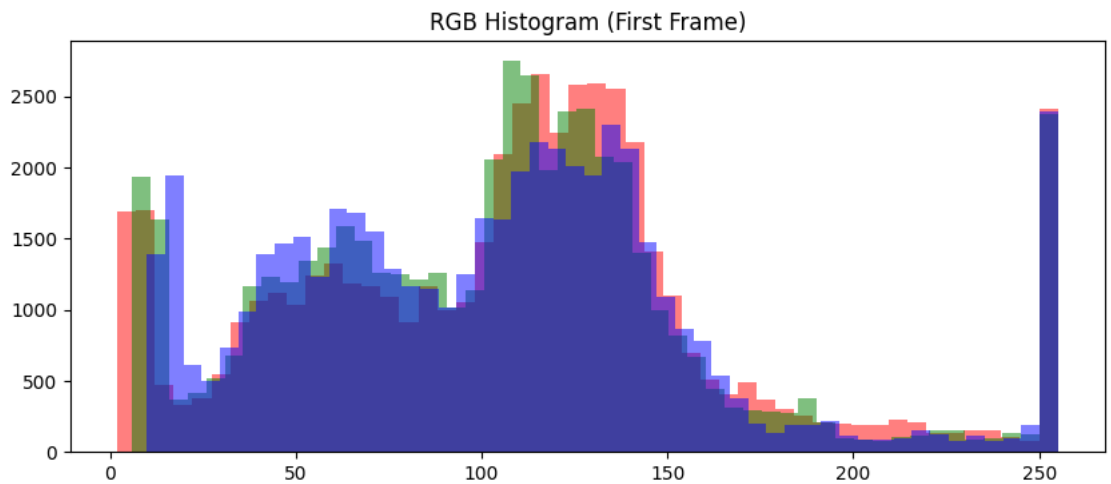
```python
# Display all extracted frames in one long horizontal strip

num_frames = len(extracted_frames)

plt.figure(figsize=(40, 4))  # Wide figure

for i in range(num_frames):
    plt.subplot(1, num_frames, i + 1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i+1}", fontsize=8)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```python
# ==============================

import matplotlib.pyplot as plt
from collections import Counter
```

```python
def analyze_rgb(frames):
    avg_r, avg_g, avg_b = [], [], []

    for frame in frames:
        avg_r.append(np.mean(frame[:,:,0]))
        avg_g.append(np.mean(frame[:,:,1]))
        avg_b.append(np.mean(frame[:,:,2]))

    return avg_r, avg_g, avg_b

avg_r, avg_g, avg_b = analyze_rgb(extracted_frames)

# Plot RGB trends
plt.figure(figsize=(10,4))
plt.plot(avg_r, label='Red Channel')
plt.plot(avg_g, label='Green Channel')
plt.plot(avg_b, label='Blue Channel')
plt.title("RGB Channel Intensity Across Frames")
plt.xlabel("Frame Number")
plt.ylabel("Average Pixel Intensity")
plt.legend()
plt.show()



plt.figure(figsize=(10,4))
colors = ['r', 'g', 'b']

for i, color in enumerate(colors):
    plt.hist(extracted_frames[0][:,:,i].ravel(), bins=50, color=color,
alpha=0.5)

plt.title("RGB Histogram (First Frame)")
plt.show()



processed_frames = preprocess_input(extracted_frames)
predictions = model.predict(processed_frames, verbose=0)

frame_labels = []
frame_confidences = []

for i, pred in enumerate(predictions):
    decoded = decode_predictions(np.expand_dims(pred, axis=0), top=1)
    label = decoded[0][0][1]
```

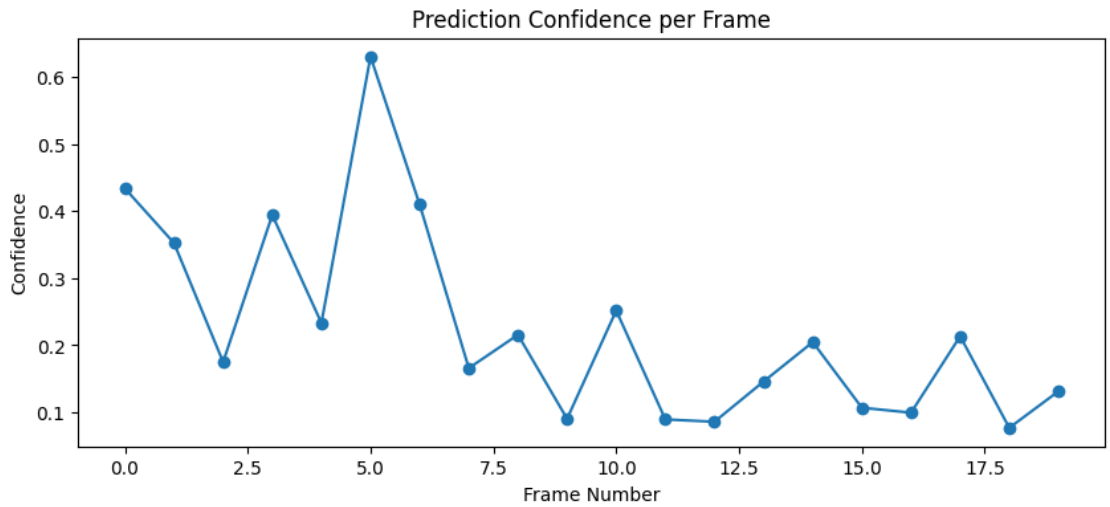
```
    confidence = decoded[0][0][2]

    frame_labels.append(label)
    frame_confidences.append(confidence)

    print(f"Frame {i+1}: {label} ({confidence*100:.2f}%)")


# ------------------------------


plt.figure(figsize=(10,4))
plt.plot(frame_confidences, marker='o')
plt.title("Prediction Confidence per Frame")
plt.xlabel("Frame Number")
plt.ylabel("Confidence")
plt.show()


# ------------------------------


most_common_label = Counter(frame_labels).most_common(1)[0][0]

print("\nMajority Voting Prediction:", most_common_label)


# ------------------------------


avg_prediction = np.mean(predictions, axis=0)
top_predictions = decode_predictions(np.expand_dims(avg_prediction, axis=0),
top=5)

print("\nTop 5 Overall Predictions:")
for pred in top_predictions[0]:
    print(f"{pred[1]} : {pred[2]*100:.2f}%")

print("\nFinal Weighted Prediction:", top_predictions[0][0][1])
```






```
Frame 1: cab (43.38%)
Frame 2: streetcar (35.24%)
Frame 3: cab (17.51%)
Frame 4: cab (39.42%)
Frame 5: cab (23.32%)
Frame 6: cab (63.02%)
Frame 7: cab (40.97%)
Frame 8: limousine (16.59%)
Frame 9: cab (21.56%)
Frame 10: streetcar (9.04%)
Frame 11: cab (25.17%)
Frame 12: maze (8.95%)
Frame 13: streetcar (8.60%)
Frame 14: jinrikisha (14.60%)
Frame 15: unicycle (20.47%)
Frame 16: jinrikisha (10.69%)
Frame 17: jinrikisha (9.95%)
Frame 18: cab (21.34%)
```

```
Frame 19: maze (7.70%)
Frame 20: garbage_truck (13.14%)
```




```
Majority Voting Prediction: cab

Top 5 Overall Predictions:
cab : 17.92%
traffic_light : 6.14%
streetcar : 5.89%
jinrikisha : 3.63%
limousine : 3.56%

Final Weighted Prediction: cab
```